

Timetable Generator

Pipeline of the Proposed Solution

A Constraint-Programming Based University Timetable Scheduling System

Submitted for Project Viva • September 2026

1. Problem Statement

Every semester, a college has to fit hundreds of theory classes, labs and tutorials into a fixed weekly bell schedule, without any teacher, room or student batch being double-booked. Done by hand, this takes a coordinator anywhere from a few days to a few weeks, and a single change — a new elective, a teacher going on leave — can force large parts of the schedule to be redone. The goal of this project is to remove that manual effort: given a department's subjects, teachers, rooms and bell schedule, the system should produce a complete, conflict-free timetable automatically, in minutes rather than days, and explain itself clearly when a request genuinely cannot be satisfied.

The hard part of this problem is not any single rule — it is that dozens of rules apply simultaneously and interact. A teacher cannot be in two classrooms at once; a lab needs three contiguous periods, not three scattered ones; a shared teacher who already has a published timetable in one department cannot be re-booked while generating another department's timetable. The proposed solution treats this as what it actually is — a constraint satisfaction problem — rather than trying to hand-write a scheduling heuristic that would need to be re-tuned for every new rule.

2. Choice of Approach: Constraint Programming (CP-SAT)

Timetabling problems are commonly solved with one of four families of technique: Integer Linear Programming (ILP), Constraint Programming (CP), Genetic Algorithms (GA), or greedy heuristics. Each was evaluated against this project's actual requirements before implementation began:

Approach	Verdict for this project
Greedy heuristic	Fast but gives no quality guarantee and no way to explain a failure. Rejected outright.
Genetic Algorithm	Scales well but needs per-college hyperparameter tuning and cannot prove a schedule is impossible — it can only fail to find one.
Integer Linear Programming	Mathematically clean, but this problem is overwhelmingly made of disjunctive rules ("these two things cannot overlap"), which ILP expresses only indirectly through big-M formulations.
Constraint Programming (CP-SAT)	Disjunctive scheduling is exactly what CP-SAT is built for. It runs a portfolio of search strategies internally (no manual tuning), and — critically — when no valid timetable exists, it proves infeasibility instead of merely giving up. That proof is what powers the diagnostic messages described in Section 4.6.

The system is built on **Google OR-Tools' CP-SAT solver**. Every class-to-be-scheduled becomes a boolean/interval decision variable; every hard rule (no clashes, valid lab blocks, workload caps) becomes a constraint over those variables; every soft preference (fewer gaps, fair workload, subjects clustered sensibly) becomes a weighted term the solver tries to maximise once all hard constraints are satisfied. This is implemented in `backend/app/solver/model_builder.py`.

*One further design rule shapes the whole system: **the solver is the only component that is ever allowed to emit a schedule**. An AI assistant is used elsewhere in the product (Section 5), but it can only propose data changes for a human to review — it can never generate a timetable itself. An AI-authored schedule would need exactly the same verification as a solver-authored one, without any of the mathematical guarantees, so that path was deliberately closed off.*

3. End-to-End Pipeline

A single 'Generate' request passes through eight stages, from raw institutional data to a published, exportable timetable. The diagram below traces that path exactly as implemented in `backend/app/routers/timetable.py` and `backend/app/solver/`.



Boxes 4 and 7 (in green) are the two points where the system checks itself before doing further work or committing a result — this is what lets a failure be reported as “Prof. Sharma is already overloaded by 4 hours” instead of a bare “INFEASIBLE”, and what catches a solver bug before a broken timetable is ever published.

4. Stage-by-Stage Description

4.1 Request intake

A department admin submits `POST /api/timetable/generate` with a scope (all sections / one year / specific sections) and a scope mode. The request is recorded as a `TimetableRun` row with status *pending* and handed to a background task, so the HTTP request returns immediately instead of blocking on the solve.

4.2 Weight resolution

Administrators never enter numeric weights. In the UI they pick plain-language importance levels (e.g. “Very Important”, “Nice to Have”) for each soft rule. `solver/weights.py` maps these onto the numeric coefficients the objective function actually uses, and a small number of rules are promoted to hard constraints rather than soft ones when marked essential.

4.3 Data loading

`solver/data_loader.py` is the single place that touches both the SQL ORM and the solver's own typed data structures. It reads sections, subjects, teachers, rooms and constraint rules for the requested scope and assembles a self-contained `ProblemData` object. Bell-schedule slots are converted into an indexed calendar (`solver/calendar.py`) so that “is this a valid 3-period contiguous block for a lab” becomes a simple index lookup rather than a date/time calculation repeated everywhere it is needed.

4.4 Pre-flight diagnostics

Before any solving is attempted, `solver/diagnostics.py` checks for structurally unsolvable situations — a lab with no batches assigned, a subject with no qualified teacher — and fails the run immediately with the exact missing resource named. This avoids spending solver time on a request that was never going to succeed, and avoids the unhelpful generic “no solution found” message.

4.5 Model building (CP-SAT encoding)

`solver/model_builder.py` turns every class that needs scheduling into CP-SAT interval and boolean variables, then adds: no-overlap constraints per teacher, per room and per student batch; valid contiguous-block constraints for labs; weekly/daily/continuous teaching-hour caps; elective co-scheduling (multiple elective options sharing one timetable slot); and the soft objective built from the resolved weights in 4.2.

4.6 Solving

`solver/engine.py` runs CP-SAT with a configurable time budget and worker count, and a fixed random seed so a given input reproduces the same output. If more than one alternative timetable is requested, each subsequent solve adds a “no-good cut” that forbids repeating the exact previous assignment and bounds the objective within roughly 15% of the best found, producing genuinely different but still near-optimal alternatives.

4.7 Independent re-validation

Before a run is ever marked *completed*, the persisted result is re-checked by `solver/validator.py` — a validator written independently of the model builder, on the same principle as double-entry bookkeeping. If it finds even one clash the model builder's constraints should have prevented, the run is force-failed rather than published. This is the project's main defence against a bug in the solver encoding silently producing a bad schedule.

4.8 Publish, edit and export

A validated run can be published, compared against an earlier version, edited slot-by-slot (which creates a new child run rather than mutating history), or exported as Excel, PDF or an .ics calendar file for personal calendar apps. Each user (admin, HOD, teacher, student) sees only the department-scoped, published view relevant to them.

5. Where AI Fits (and Where It Deliberately Does Not)

An optional LLM integration (any OpenAI-compatible endpoint) assists with three things, all outside the actual scheduling decision: turning a plain-English constraint description into the same rule catalogue the UI uses; proposing bulk data edits (e.g. “increase every teacher's daily cap by 1”) that an admin must explicitly review and apply; and rephrasing an already-computed infeasibility explanation in friendlier language. In every one of these, the underlying fact (the rule, the data change, the cause of failure) is computed deterministically first, and the model is only ever restating something already verified — never inventing it. Without an API key configured, all three features degrade gracefully and the rest of the system is unaffected.

6. Technology Stack

Layer	Technology	Purpose
Scheduling engine	Google OR-Tools (CP-SAT)	Constraint model, search, infeasibility proofs
Backend API	FastAPI + SQLAlchemy + Alembic	REST endpoints, ORM, schema migrations
Database	SQLite (dev) / PostgreSQL (prod)	Institutional and generated-schedule data
Auth	JWT + bcrypt	Role-based access (admin / HOD / teacher / student)
Frontend	React 18 + TypeScript + Vite + Tailwind	Setup wizard, timetable viewer, admin tools
Export	openpyxl, reportlab, icalendar	Excel, PDF and calendar (.ics) output
AI assist (optional)	Any OpenAI-compatible LLM API	NL constraint parsing, bulk-edit proposals only

7. Conclusion

The proposed pipeline treats timetabling as what it mathematically is — a large constraint satisfaction problem — rather than approximating it with a heuristic. Two independent checks (pre-flight diagnostics before solving, and an independently-written validator after solving) mean the system either produces a schedule it can prove is correct, or explains precisely why it cannot, rather than silently returning a flawed timetable.